

PPAW - Laborator 10

Objective

1. Nivel Servicii (sau Business Layer)
2. Dependency Injection

1. Nivel Servicii (sau Business Layer)

Exercițiu 1: Implementați un nou nivel al aplicației numit NivelServicii (sau BusinessLayer) care sa fie situat între PresentationLayer si DataAccess. Mutați logica mai complexa a aplicației în acest nivel. Urmăriți aplicația demo de la curs și citiți mai multe detalii despre Business Layer în cadrul articolelor [1] și [2].

2. Dependency Injection

Exercițiu 2: Înlocuiți referințele directe către NivelAccessDate și către BusinessLayer (sau cele obținute cu ajutorul clasei StocareFactory) cu modulul de injectare a dependentelor ([3], [4]).

Exemplu etape pentru .Net:

1. Adăugați utilizând NuGetManager o referință către pachetul Autofac.WebApi
2. Creați un folder special (Infrastructure) în care creați clasa ContainerConfigurer cu următorul conținut:

(Observație: Secțiunea marcată cu galben trebuie personalizată conform entităților din proiectul propriu.)

```
public class ContainerConfigurer
{
    public static void ConfigureContainer()
    {
        var builder = new ContainerBuilder();

        // Register dependencies in controllers
        builder.RegisterApiControllers(Assembly.GetExecutingAssembly());

        // Register individual types
        builder.RegisterType<CarStoreDbContext>()
            .As<ICarDbContext>();

        builder.RegisterType<CompaniesService>()
            .As<ICompanies>();

        // Get your HttpConfiguration.
        var config = GlobalConfiguration.Configuration;

        // Set the dependency resolver to be Autofac.
        var container = builder.Build();
        config.DependencyResolver = new AutofacWebApiDependencyResolver(container);
    }
}
```

```
}  
}
```

3. Adăugați apelul de mai jos in cadrul metodei `Application_Start()` din cadrul fișierului `Global.asax.cs`

```
ContainerConfigurer.ConfigureContainer();
```

4. Adăugați o interfață `ICarDbContext` si modificați clasa `CarStoreDbContext` sa implementeze aceasta interfață

```
public interface ICarDbContext  
{  
}
```

```
public class CarStoreDbContext : DbContext, ICarDbContext  
{  
...  
}
```

5. Adăugați o proprietate de tipul interfeței si adăugați un constructor care primește ca parametru un obiect de tipul interfeței si îl atribuie proprietății interne:

```
private ICarDbContext db;  
  
public CompaniesService(ICarDbContext db)  
{  
    this.db = db;  
}
```

Exercițiu 3: Testati diferite setari pentru a stabili durata de viata a obiectelor injectate (Singleton, Scoped, Transient). Postati pe classroom un document cu modul de comutare intre cele 3 valori si diferentele intre ele.

Bibliografie

1. <https://www.codemag.com/Article/1411071/The-Mythical-Business-Layer>
2. <https://www.thereformedprogrammer.net/architecture-of-business-layer-working-with-entity-framework-core-and-v6-revisited/>
3. Autofac integration with MVC: <https://autofaccn.readthedocs.io/en/latest/integration/mvc.html>
4. Autofac integration with Web API: <https://autofaccn.readthedocs.io/en/latest/integration/webapi.html>